Design

Simple data types for basic markdown objects are created and composed into ADT. Parsing is done by creating parsers for basic data types, composed into the `markdownParser`. Printing to `html` is done by creating a type class `PrettyBoy` with a method `prettyPrint`. Markdown data types are instances of `PrettyBoy`, with `prettyPrint` defined accordingly.

Parsers and Parser Combinators

`pUntil` takes two parsers, and repeatedly performs the first parser until the first time the second parser succeeds (discarding the result of the second parser). Parser was implemented by combining recursion, using <|> to either end recursion (with base case) or take one step forward and repeat. <$> and <*> were then utilised in the form of `liftA2`, concatenating parsed characters into a string, regardless of length. Parser combinator was used directly to make many parsers (like `pImage` and `pSubScript`) and also combined into the `pBetween`, used in `pBold`, among other parsers.

`pTable` combines `pUntil` with a `many` function, allowing for each cell to end with a different amount of inline whitespace characters. Further, >>= is used to enforce the context-sensitive requirement that each row have the same number of cells. a do block is used to "unwrap" the first row, and its length is used in a `guard` clause to assert that all other rows adhere to the same width requirement. a Similar technique is used in `pHeading`, parsing #s and asserting that the length is at most 6.

`pad` was used to ensure all `Block` types are properly related to each other. Hence, `Block` types do not consume any whitespace before or after, All of that is handled by `pad`. As a result, many features do not need to be implemented explicitly. For example, the requirement that ending triple back-ticks and alternate headings need to be on a line by themselves. `pad` handles this implicitly by asserting that `Block`s must have at least 1 newline between them.

Printing to html

`PrettyBoy` has a minimal definition `unwrapPrint`, specifying how an algebraic data types should be unwrapped and printed. From there, the function `prettyPrint` is given for free. This reduces repetition between printing for different data types. If more granularity is required, it can be done by manually setting the `PrettyBoy` functions `preText`, `postText` and `prettyPrintAux`. Thus Printing is handled recursively by a single call of `prettyPrint`.

Thus, code is incredibly flexible:

- Changing default indentation can be done by passing different delimiters to `prettyPrint`. e.g. passing `"\n\t"` produces an extra indentation through out the whole html doc.
- Changing specific print features is easy, by changing the `unwrapPrint` function of the specific type you want to change.
- Adding new features is easy, by creating a new type instance of `PrettyBoy`, defining a parser for that type, composing with `markdownParser`, and defining `unwrapPrint`. Minimal changes are needed to the existing codebase.

An alternative approach was setting `ADT` as an instance of functor, applicative, and monad, then defining small printing functions and composing them into the `convertADTHTML` function. This method would require much more abstraction and set up, and be even more modular than my current setup. Current approach was selected is more concrete and specialised to the problem at hand, allowing me to develop solutions to the specific task.

IO handling

I add html elements: an title input box and a save button. HTML elements are connected to an observable which emits a signal every time said element is interacted with. I process the signals into `Actions`, and merge them. I compose the `reset` action to happen before any other action, ensuring clean States each time. I modify the API request to contain (a) whether to save the file (b) the title and (c) the markdown content. I perform intermediary parsing of the signal in `main.hs`, and perform actions (updating titles, saving files), in accordance with the received signal.

Additional Features

Nested text modifiers were implemented by allowing `ModText` to have another instance of `ModText` inside. The only adjustment was changing `pBold` (and other text modifiers) to run `pModText` inside (creating a co-recursive structure). Printing nested modifiers is automatically handled since all types are already instances of `PrettyBoy`. Implementation is interesting because it relies on co-recursion to terminate. Since types are cleanly separated, it is easy to assert that some `ModText` types do not allow further recursion (most explicitly seen in `Untouched`), and since `<|>` prevents the same combinations of parsers to be repeated, termination (success or fail) is guaranteed .

Additional text modifiers were added, i.e. `//subscript\` and `\superscript//`. Feature demonstrates how extensible code is, even with different closing and opening tokens, with different lengths, `pUntil` handles it all succinctly. Printing is also simple, defining the types as instances of `PrettyBoy` with minimal definition. Furthermore, additional text modifiers showcase how robust the co-recursive text modifier system is. The only adjustment necessary was extending `ModText` to include superscript and subscript.